

VulScan: A Deep Learning-based Vulnerability Scanning Tool

Dev Saini¹, Dr. Vivek Chaturvedi²
Indian Institute of Technology, Palakkad

User Manual

About: A Python-based tool that detects software vulnerabilities in the source code files of IoT operating systems. The tool is primarily focused on detecting C/C++-based vulnerabilities.

Vulnerabilities Detected: VulScan majorly detects 36 types of vulnerabilities; the CWE-id/description of all these vulnerabilities is available at the end of this manual [\[1\]](#).

Specifications:

- All platform tool, executables available for Linux, Windows, as well as MacOS.
- End-to-end working with no interference from the user (except supplying input).
- Can scan a single source code file or multiple files together for vulnerabilities.
- Generates a well-documented vulnerability report describing the analysis of the file given for vulnerability check.

System Requirements:

- Ubuntu Operating System (Version 20.04+)
- Python (Version 3.8.10+)
- 3 GB HDD/SSD storage space (to store the executable).

Tool Description: VulScan is an all-platform executable that can be downloaded into the system and directly used to detect vulnerabilities. It is an end-to-end black box executable wherein the user needs to supply the input and the tool directly generates the vulnerability report without any further user action.

- **Input:** A single C or C++ source code file/Directory containing C or C++ source code files.
- **Output:** **vulscan_report_<file_name>.pdf** (document enveloping detected vulnerabilities and the pre-processed source code file line-by-line for user reference).

NOTE: This report is generated separately for each file supplied to VulScan.

Tool Usage

- Unzip “**vulscan.zip**” in a suitable directory of your choice to extract the executable file for the VulScan tool.
- To run the executable, open the terminal window at the directory path where the tool was extracted in the previous step (**<DIR_PATH>**) and press enter to run the executable using the command below (Ubuntu). Wait for some time until the executable unpacks the tool and an input interface pops up.

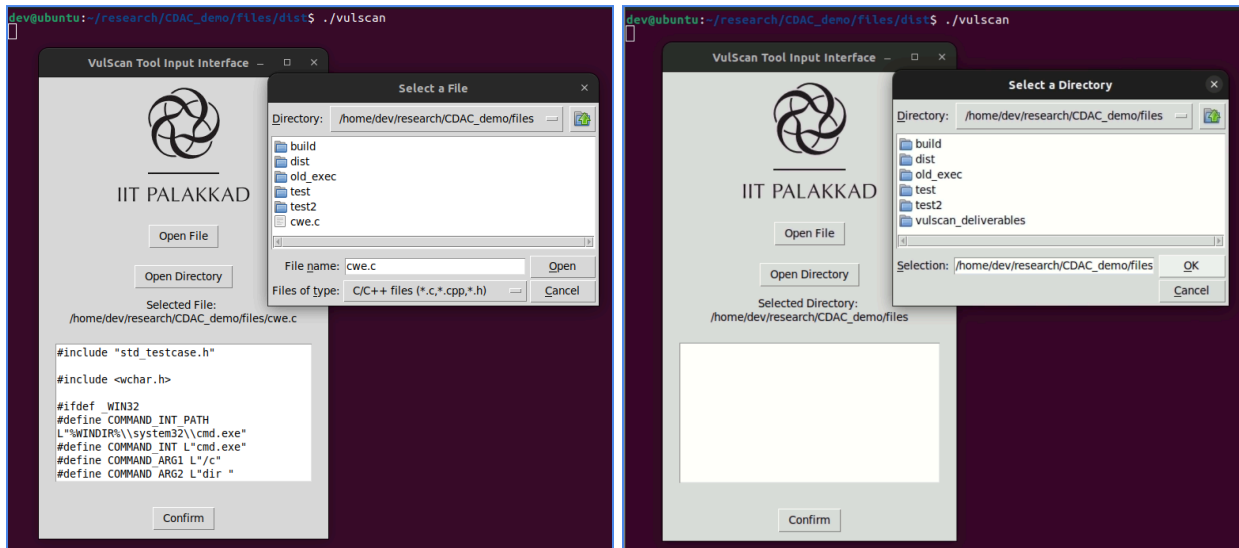
```
<DIR_PATH> $ ./vulscan
```

```
dev@ubuntu:~/research/CDAC_demo/files/dist$ ./vulscan
```

- Next, from the VulScan input interface (refer [here](#)), select the file or the entire directory (containing multiple source code files), which has to be checked for vulnerabilities. Press ‘**Confirm**’ to start vulnerability detection.
- Please wait until the tool scans for vulnerabilities and generates the report.

NOTE:

- In the case of a directory selection, double-click the directory to open it, and then press ‘Ok’ to select the directory.
- In the case of a single input file, VulScan generates the vulnerability report (PDF) in the same directory as the file input to the tool.
- In the case of a directory, VulScan generates the vulnerability reports (PDF) for all the source code files in the directory, inside this particular directory.



a) Selecting a file (name: cwe.c)

b) Selecting a directory (name: files)

VulScan Report

The vulnerability report generated by VulScan is a comprehensive report that envelopes all the vulnerabilities detected. The report represents these vulnerabilities as the line of code detected as vulnerable, along with the type of vulnerability that line contains.

The tool also attaches the preprocessed source code file to the report (refer [here](#)), which it uses to scan the original source code file line-by-line. This text file attached in the PDF contains the input source code file represented as each line of code, along with the corresponding line numbers. This attached file provides ease of access to the tool user to refer to the detected vulnerable line directly, rather than scanning the original input source code file.

Below is a description of the report's contents (colors are referred to from the report):



VulScan Vulnerability Results

File Name: `/home/dev/research/CDAC_demo/files/cwe.c`

Please find below the vulnerable lines identified along with their corresponding vulnerability id/description. Line numbers mentioned in RED can be referred in the formatted line-by-line input file attached with this report.

1. CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer

Line 27: `wchar _tdatabuffer [ 100 ] = command _ arg2 ;`

For more info regarding the vulnerability, please visit:

<https://cwe.mitre.org/data/definitions/119.html>

2. CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')

Line 77: `wscat ( data, 1 " *. * " ) ;`

For more info regarding the vulnerability, please visit:

<https://cwe.mitre.org/data/definitions/120.html>

- Name of the source code file processed
- Vulnerability CWE/description
- Line of code where this vulnerability was detected
- URL (if available) to get more information regarding the vulnerability

File Name: `/home/dev/research/CDAC_demo/files/cwe.c`

Input file formatted line-by-line for developer reference below.

```
1. #ifdef _WIN32
2. #define COMMAND_INT_PATH L"%WINDIR%\\system32\\cmd.exe"
3. #define COMMAND_INT L"cmd.exe"
4. #define COMMAND_ARG1 L"/c"
5. #define COMMAND_ARG2 L"dir "
6. #define COMMAND_ARG3 data
7. #define COMMAND_INT_PATH L"/bin/sh"
8. #define COMMAND_INT L"sh"
9. #define COMMAND_ARG1 L"-c"
10. #define COMMAND_ARG2 L"ls "
```

- Text file containing source code file formatted line-by-line (this is necessary since the line numbers detected above are new line numbers after the input file had been preprocessed to remove comments and duplicates).

VulScan Limitations:

- VulScan only helps detect possibly vulnerable lines of code that may turn into a backdoor for any malicious entity or hacker to exploit the software code. The tool currently does not propose any viable fix for the detected vulnerability.
- VulScan currently detects only 36 types of vulnerabilities mentioned in the document above.
- Some vulnerabilities detected by VulScan can be treated as false positives, implying that they may not have a serious impact on the security of the source code file.

[Continue to Vulnerabilities List...](#)

VulScan Vulnerabilities List

1. TOCTOU (Time Of Check, Time Of Use) vulnerability.
2. Arguments to a function call should be checked to ensure that it does not come from an untrusted source, first verify that it contains nothing dangerous.
3. Buffer size must be `_MAX_PATH+1` or larger for this function to be safe.
4. CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer.
5. CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow').
6. CWE-126: Buffer Over-read.
7. CWE-134: Use of Externally-Controlled Format String.
8. CWE-190: Integer Overflow or Wraparound.
9. CWE-20: Improper Input Validation.
10. CWE-250: Execution with Unnecessary Privileges.
11. CWE-327: Use of a Broken or Risky Cryptographic Algorithm.
12. CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition').
13. CWE-377: Insecure Temporary File.
14. CWE-676: Use of Potentially Dangerous Function.
15. CWE-732: Incorrect Permission Assignment for Critical Resource.
16. CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection').
17. CWE-807: Reliance on Untrusted Inputs in a Security Decision.
18. Check buffer boundaries if calling a function in a loop and make sure you are not in danger of writing past the allocated space.
19. Check to be sure that arguments passed to a function call will not copy more data than can be handled, resulting in a buffer overflow.
20. Check to be sure that the format string passed as an argument to a function call does not come from an untrusted source that could have added formatting characters that the code is not prepared to handle. Additionally, the format string could contain ``%s'` without precision that could result in a buffer overflow.
21. Check to be sure that the non-constant format string passed as arguments to a function call does not come from an untrusted source that could have added formatting characters that the code is not prepared to handle.
22. Consider using `strlcat()` instead.
23. DNS results can easily be forged by an attacker (or arbitrarily set to large values, etc), and should not be trusted.
24. Don't use on memory intended to be secure, because the old structure will not be zeroed out.



25. Environment variables are highly untrustworthy input. They may be of any length and contain any data. Do not make any assumptions regarding content or length. If at all possible avoid using them, and if it is necessary, sanitize them and truncate them to a reasonable length.
26. Extra care should be taken to ensure that character arrays that are allocated on the stack are used safely. They are prime targets for buffer overflow attacks.
27. Gets is unsafe. No bounds checking is performed, and the buffer is easily overflowable by the user. Use fgets(buf, size, stdin) instead.
28. Impersonation functions return error codes when they fail. These error codes must be checked; otherwise, code could be run with extra privileges when an impersonation has failed.
29. LoadLibrary will search several places for a library, if no path is specified, allowing trojan DLLs to be inserted elsewhere even if the intended DLL is correctly protected from overwriting. Make sure to specify the full path.
30. Many calls for generating temporary file names are insecure (susceptible to race conditions). Use a securely generated filename, for example, by pulling 64 bits of randomness from <file_name>, base 64 encoding it and using that as a file suffix.
31. Many program execution commands under Windows will search the path for a program if you do not explicitly specify a full path to the file. This can allow trojans to be executed instead. Also, be sure to specify a file extension, since otherwise multiple extensions will be tried by the operating system, providing another opportunity for trojans.
32. Standard random number generators should not be used to generate randomness used for security reasons. For security sensitive randomness a cryptographic randomness generator that provides sufficient entropy should be used.
33. This function can throw exceptions in low memory conditions. Use 'InitialCriticalSectionAndSpinCount' instead.
34. Truncate all input strings to a reasonable length before passing them to this function.
35. When setting signal handlers, do not use the same function to handle multiple signals.
36. There exists the possibility a race condition will result if 2 or more different signals are sent to the process at nearly the same time. Also, when writing signal handlers, it is best to do as little as possible in them. The best strategy is to use the signal handler to set a flag, that another part of the program tests and performs the appropriate action(s) when it is set.